

2D Non-Hydrostatic Atmospheric Model

Implementation, Benchmarking & Exponential Time Integration

Comprehensive Technical Report

Alen Veliyathuparamban
alen.veliyathuparamban@ucdconnect.ie
Supervisor: Dr. Colm Clancy
University College Dublin — June 2026

Giraldo & Restelli (2008) J. Comput. Phys. 227:3849–3877 | Pudykiewicz & Clancy (2022) J. Comput. Phys. 449:110803

1. Project Overview

This project implements a 2D non-hydrostatic atmospheric model solving the compressible Euler equations in theta-pi perturbation form (Equation Set 1 of Giraldo & Restelli 2008). Nine time integration schemes are implemented and benchmarked against two published test cases: the Rising Thermal Bubble (G&R 2008, Case 2) in a 1 km x 1 km domain, and the Convective Bubble (Pudykiewicz & Clancy 2022, Experiment 1) in a 5 km x 5 km domain. The central research question is whether exponential propagation integrators (EPI) with a **full Jacobian** $J_n = dF/du$ can reproduce correct bubble dynamics at large timesteps ($dt = 15$ s, acoustic CFL ~ 260), as demonstrated in P&C (2022).

1.1 Schemes Overview

Scheme	Type	Order	dt (G&R)	Acoustic CFL	Key feature
FTCS	Forward Euler	1st	0.01 s	0.35	Simplest; CFL<1 required
BTCS	Heun (Predictor-Corrector)	2nd	0.01 s	0.35	2 RHS calls/step
CTCS	Leapfrog	2nd	0.01 s	0.35	Robert-Asselin filter needed
RK4	Runge-Kutta 4	4th	0.01 s	0.35	Reference; 4 RHS calls/step
SI	Semi-implicit IMEX	1st	1.0 s	34.7	L implicit via GMRES
EPI2	Krylov (const. L)	2nd	15 s	520	$\exp(L \, dt)$; advection explicit
EPI3	Krylov (const. L)	3rd	15 s	520	EPI2 + ϕ_2 correction
EPI2FJ	Krylov (full J_n)	2nd	15 s	260*	$J = dF/du$; advection in exp
EPI3FJ	Krylov (full J_n)	3rd	15 s	260*	EPI2FJ + ϕ_2 correction

* P&C; domain $dx=20$ m, $dt=15$ s: acoustic CFL = $347 \cdot 15 / 20 = 260$.

2. Mathematical Formulation

2.1 Governing Equations

The model solves the 2D compressible Euler equations in **theta-pi perturbation form** (Equation Set 1 of G&R 2008). Every variable is split into a hydrostatically balanced base state (overbar) and a perturbation (prime):

$$du/dt = -u*du/dx - w*du/dz - c_p*theta_bar*d(pi')/dx - c_p*theta'*d(pi')/dx$$

$$dw/dt = -u*dw/dx - w*dw/dz - c_p*theta_bar*d(pi')/dz + g*theta'/theta_bar - c_p*theta'*d(pi')/dz$$

$$d(pi')/dt = -(R_d/c_v)*pi_bar*(du/dx+dw/dz) + g*w/(c_p*theta_bar) - u*d(pi')/dx - w*d(pi')/dz - (R_d/c_v)*pi'*(du/dx+dw/dz)$$

$$d(theta')/dt = -u*d(theta')/dx - w*d(theta')/dz - w*d(theta_bar)/dz$$

Variables: u, w = horizontal and vertical velocities [m/s]; theta' = potential temperature perturbation [K]; pi' = Exner pressure perturbation [-]. The base-state profiles theta_bar(z) and pi_bar(z) are described in Section 2.3.

2.2 L + N Splitting

For the semi-implicit and EPI schemes, the RHS $F(q) = L*q + N(q)$ is split into:

$$F(q) = L*q + N(q)$$

Part	Content	Stiffness
L*q (Linear)	Acoustic pressure gradient: $-c_p*theta_bar*grad(pi')$; Buoyancy: $g*theta'/theta_bar$	HIGH — speed of coupling: $c \sim 347$ m/s
N(q) (Nonlinear)	Advection: $-u*grad(q)$; Perturbation products: $-c_p*theta'*grad(pi')$, $-pi'*div(u)$	LOW — advection speed ~ 2 -10 m/s

L is a **constant** linear operator when the base state is isentropic (theta_bar = const). This is exploited by EPI2/EPI3 (const. L) which precompute $\exp(L*dt)$ once. EPI2FJ/EPI3FJ instead use $J_n = dF/du$ evaluated at each timestep u^n (see Section 5).

2.3 Base State

Default: ISENTROPIC (neutral stratification) — used in ALL experiments:

Quantity	Expression	Value / Notes
theta_bar(z)	$T_0 = \text{constant}$	300 K everywhere ($dtheta_bar/dz = 0$)
pi_bar(z)	$1 - g*z/(c_p*T_0)$	Linear decrease; $pi_bar(0) = 1.000$, $pi_bar(1000m) \sim 0.9968$
dtheta_bar/dz	0	No restoring force -> bubble rises freely
$c_s = \sqrt{\gamma*R_d*T_0}$	$\sqrt{1.4*287*300}$	~ 347 m/s (constant)
N (Brunt-Vaisala)	0	Neutral atmosphere; no gravity waves

Alternative: ISOTHERMAL (stable, NOT used in experiments) — temperature $T_0 = \text{constant}$, theta_bar increases with height. $N^2 = g^2/(c_p*T_0) \sim 3.2e-4 s^{-2}$, period ~ 351 s. The bubble **OSCILLATES** instead of rising. This is wrong for bubble benchmarks; it is kept in grid.py for reference only.

2.4 Physical Constants (from src/grid.py DEFAULTS)

Constant	Symbol	Value	Units	Source
Gravitational acceleration	g	9.81	m/s ²	grid.py: DEFAULTS["g"]
Specific heat (const. p)	c_p	1004.0	J/kg/K	grid.py: DEFAULTS["cp"]
Specific heat (const. v)	c_v	717.0	J/kg/K	grid.py: DEFAULTS["cv"]
Gas constant (dry air)	R_d	287.0	J/kg/K	grid.py: DEFAULTS["Rd"]

Constant	Symbol	Value	Units	Source
Reference pressure	p_0	1.0e5	Pa	grid.py: DEFAULTS["p0"]
Reference temperature	T_0	300.0	K	grid.py: DEFAULTS["T0"]
Kappa = R_d/c_p	kappa	0.2857	-	computed in Grid.__init__
gamma = c_p/c_v	gamma	1.4	-	derived
Speed of sound	c_s	~347	m/s	$\sqrt{\text{gamma} * R_d * T_0}$

2.5 Grid (src/grid.py DEFAULTS)

Parameter	Default value	Meaning
Lx	1000.0 m	Domain width (overridden by each experiment)
Lz	1000.0 m	Domain height (overridden by each experiment)
dx	10.0 m	Horizontal grid spacing
dz	10.0 m	Vertical grid spacing (always = dx)
diffusion_coeff	0.0 m ² /s	Explicit diffusion kappa (0 = no diffusion)
stratification	"isentropic"	Base state type
sponge_fraction	0.20	Top 20% of domain is Rayleigh sponge layer
sponge_strength	0.01 s ⁻¹	Max damping coefficient alpha in sponge
Grid type	Unstaggered	All variables at same (nz x nx) cell-centre points

Cell centres: $x_i = (i+0.5)*dx$ for $i=0..nx-1$; $z_k = (k+0.5)*dz$ for $k=0..nz-1$. The sponge applies Rayleigh damping $\alpha(z)*q$ in the top 20% ($z > 0.8*Lz$): $\alpha(z) = 0.01 * \sin^2(\pi/2 * (z-z_s)/(Lz-z_s))$.

3. Numerical Schemes — Full Detail

3.1 FTCS (Forward Euler)

$$q^{n+1} = q^n + dt * F(q^n)$$

1st-order accurate in time. $F = L*q + N(q) + \kappa*\nabla^2*q$. Stability requires acoustic CFL = $c_s*dt/dx < 1$. At $dx=10$ m: $dt_{max} = 10/347 \sim 0.029$ s. Used at $dt=0.01$ s (CFL=0.35). Cost: 1 RHS evaluation per step. 70,000 steps for G&R; Case 2.

3.2 BTCS (Heun's Method / 2-Stage Predictor-Corrector)

Note: Despite the name "BTCS" (Backward-Time Centred-Space), this is actually Heun's method — a 2-stage explicit Runge-Kutta. It is NOT backward Euler.

$$q^* = q^n + dt * F(q^n) \text{ [Euler predictor]}$$

$$q^{n+1} = q^n + (dt/2) * (F(q^n) + F(q^*)) \text{ [trapezoidal corrector]}$$

2nd-order accurate. Same stability as FTCS: CFL < 1 required. Cost: 2 RHS evaluations per step. Used at $dt=0.01$ s (CFL=0.35).

3.3 CTCS (Leapfrog)

$$q^{n+1} = q^{n-1} + 2*dt * F(q^n)$$

2nd-order accurate, centred in both space and time. Requires two time levels. First step bootstraps with FTCS (q^{-1} not available). Without a filter, the computational mode (pi-mode) decouples from the physical mode and grows. The **Robert-Asselin filter** ($\alpha=0.1$) damps this:

$$q^n_{filt} = q^n + \alpha*(q^{n-1} - 2*q^n + q^{n+1}) \quad \alpha=0.10$$

Applied AFTER computing q^{n+1} but BEFORE the next step. The filter slightly reduces accuracy but prevents computational instability. Used at $dt=0.01$ s. Cost: 1 RHS evaluation per step (+ filter cost).

3.4 RK4 (Classical 4th-Order Runge-Kutta) — Primary Reference Scheme

$$k1 = F(q^n)$$

$$k2 = F(q^n + (dt/2)*k1)$$

$$k3 = F(q^n + (dt/2)*k2)$$

$$k4 = F(q^n + dt*k3)$$

$$q^{n+1} = q^n + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)$$

4th-order accurate in time, 2nd-order in space (centred FD). Stability requires acoustic CFL ~ 0.3 - 0.9 . At $dx=10$ m, $dt=0.01$ s: acoustic CFL = $347*0.01/10 = 0.347$. Cost: 4 RHS evaluations per step. 70,000 steps for G&R; $t=700$ s. Wall time: ~ 250 s on a single CPU core.

Diffusion: For the 1 km G&R; domain at $dx=10$ m, $\kappa=0$ (inviscid). For the 5 km P&C; domain at $dx=20$ m, $\kappa=2$ m²/s is added to damp $2*dx=40$ m noise: $\tau_{noise} = (2*dx)^2/(\pi^2*\kappa) = 1600/(9.87^2) \sim 81$ s; $\tau_{bubble} = a^2/\kappa = 400^2/2 = 80,000$ s $\gg 1800$ s (bubble unaffected).

3.5 SI (Semi-Implicit IMEX, 1st Order)

$$(I - (dt/2)*L) q^{n+1} = (I + (dt/2)*L)*q^n + dt*N(q^n)$$

Treats the stiff linear operator L implicitly (Crank-Nicolson in L) and the nonlinear advection N explicitly (forward Euler). This removes the acoustic CFL constraint: large dt is feasible. The linear system is solved at each timestep by **GMRES** (scipy.sparse.linalg.gmres, atol=1e-10, rtol=1e-8) using a matrix-free matvec: $v \rightarrow v - (dt/2)*L*v$. The initial guess for GMRES is q^n .

Parameters used: $dt=1.0$ s, acoustic CFL = $347 \times 1/10 = 34.7$. 700 steps for G&R; Case 2. **Known issue:** blows up at $t \sim 550$ s when the mushroom cap forms. Sharp gradients at the cap edges make the explicit $N(q^n)$ term marginally unstable.

3.6 EPI2 / EPI3 (Exponential Propagation, Constant-L Approximation)

With the approximation $J_n \approx L$ (L is the constant linear operator):

$$\text{EPI2: } q^{n+1} = \exp(L \cdot dt) \cdot q^n + \phi_1(L \cdot dt) \cdot dt \cdot N^n$$

$$\text{EPI3: } q^{n+1} = \exp(L \cdot dt) \cdot q^n + \phi_1(L \cdot dt) \cdot dt \cdot N^n + (2/3) \cdot \phi_2(L \cdot dt) \cdot dt \cdot R^{n-1}$$

$$\text{where: } \phi_1(A) = (e^A - I) \cdot A^{-1}, \quad \phi_2(A) = (e^A - I - A) \cdot A^{-2}$$

$$R^{n-1} = N^{n-1} - N^n \text{ (nonlinear remainder from previous step)}$$

The ϕ -functions are computed via **Krylov subspace method** (Arnoldi iteration, $m_{\max}=30$ vectors). For large acoustic CFL ($c_s \cdot dt/dx \gg 1$), a single Krylov solve overflows (the spectral radius of $L \cdot dt$ is very large). The solution is to **sub-divide** the major step dt into p sub-steps of $h=dt/p$, choosing p so that the spectral radius per sub-step satisfies:

$$\rho(L \cdot h) = c_s \cdot (\pi/dx) \cdot h \leq 15 \Rightarrow p = \text{ceil}(c_s \cdot \pi \cdot dt / (15 \cdot dx))$$

At $dt=15$ s, $dx=10$ m: $p = \text{ceil}(347 \cdot \pi \cdot 15 / (15 \cdot 10)) = \text{ceil}(109.0) = 110$ sub-steps of $h \sim 0.14$ s. At $dt=15$ s, $dx=20$ m: $p = \text{ceil}(347 \cdot \pi \cdot 15 / (15 \cdot 20)) = \text{ceil}(54.5) = 55$ sub-steps of $h \sim 0.27$ s.

3.7 EPI2FJ / EPI3FJ (Full-Jacobian EPI)

The key innovation of P&C (2022): use the FULL Jacobian $J_n = dF/du|_{u^n}$ instead of the constant L . J_n includes both acoustics AND linearised advection. (See Section 5 for full derivation and implementation details.)

4. Code Architecture

The codebase has a clean layered structure: `grid/physics (src/)`, time integration (`src/integrators.py`), and experiments (`experiments/`).

File	Lines	Purpose
<code>src/grid.py</code>	265	Unstaggered grid, base state (isentropic/isothermal), sponge layer, physical constants
<code>src/dynamics.py</code>	284	RHS computations: <code>compute_rhs</code> , <code>compute_linear_rhs</code> , <code>compute_nonlinear_rhs</code>
<code>src/integrators.py</code>	~600	All 9 integration schemes + Shapiro filter + Krylov engine + state vec utilities
<code>src/results.py</code>	~80	Experiment result saving (npz + json)
<code>experiments/gr_case2_benchmark.py</code>	310	G&R (2008) Case 2: Rising Thermal Bubble
<code>experiments/pc_exp1_benchmark.py</code>	~480	P&C (2022) Exp 1: Convective Bubble
<code>experiments/robert_bubble.py</code>	~200	Robert (1993) smooth bubble
<code>experiments/density_current.py</code>	~200	Straka density current ($\kappa=75 \text{ m}^2/\text{s}$)
<code>experiments/efficiency_study.py</code>	~150	Wall-time comparison across schemes
<code>tests/test_integrators.py</code>	~150	Unit tests: zero-amplitude, all 9 schemes
<code>tests/test_grid.py</code>	~80	Unit tests: grid construction, base state
<code>compare_schemes.py</code>	~100	Side-by-side scheme comparison plots
<code>run_model.py</code>	~60	Simple CLI runner
<code>menu.py</code>	~120	Interactive CLI menu

4.1 `src/grid.py` — Grid Class

Class: `Grid(params: dict = None)`

The Grid class is the central data structure. It holds all coordinate arrays, physical constants, base state profiles, and the sponge layer. All other modules receive a Grid instance.

Constructor flow (`Grid.__init__`):

- Merge params dict with DEFAULTS; extract L_x , L_z , dx , dz , physical constants
- Compute $n_x = \text{int}(\text{round}(L_x/dx))$, $n_z = \text{int}(\text{round}(L_z/dz))$
- Assert $L_x == n_x * dx$, $L_z == n_z * dz$ (domain must be exactly divisible)
- Build coordinate arrays: $x_{1d} = (\text{arange}(n_x)+0.5)*dx$; z_{1d} similarly
- Build meshgrid: x_{2d} , $z_{2d} = \text{meshgrid}(x_{1d}, z_{1d})$ -- shape (nz, nx)
- `_build_base_state()`: returns π_{bar} , θ_{bar} , $d\theta_{\text{bar}}/dz$ arrays of shape (nz,)
- `_build_sponge(frac=0.20, strength=0.01)`: returns α array of shape (nz,)
- `allocate_state()`: returns $\{u, w, \theta, \pi\}$ each of shape (nz, nx) filled with zeros

4.2 `src/dynamics.py` — RHS Functions

Finite difference operators:

```
_dx(f, dx): (f[i+1] - f[i-1]) / (2*dx) -- 2nd-order centred, periodic in x via
                                                    np.roll

_dz(f, dz): (f[k+1] - f[k-1]) / (2*dz) -- 2nd-order centred interior rows
            (f[1] - f[0]) / dz -- 1st-order one-sided at k=0 (bottom)
            (f[-1] - f[-2]) / dz -- 1st-order one-sided at k=nz-1 (top)
```

Numba JIT: if numba is installed, Numba-compiled `_dx_nb/_dz_nb` replace the NumPy versions automatically on first call. These avoid temporary array allocation and are 10-50x faster on large grids. No code changes are needed — detection is automatic.

compute_linear_rhs(state, grid) — returns $L \cdot q$:

```

rhs_u = -c_p * theta_bar * d(pi')/dx
rhs_w = -c_p * theta_bar * d(pi')/dz + g * theta' / theta_bar
rhs_pi = -(R_d/c_v) * pi_bar * (du/dx + dw/dz) + g*w / (c_p*theta_bar)
rhs_theta = -w * dtheta_bar/dz (=0 for isentropic base state)
BCs enforced: rhs_w[0,:] = rhs_w[-1,:] = 0 (no flux at z=0, z=Lz)

```

compute_nonlinear_rhs(state, grid) — returns $N(q)$:

```

rhs_u = -u*du/dx - w*du/dz - c_p*theta'*d(pi')/dx
rhs_w = -u*dw/dx - w*dw/dz - c_p*theta'*d(pi')/dz
rhs_pi = -u*d(pi')/dx - w*d(pi')/dz - (R_d/c_v)*pi'*(du/dx+dw/dz)
rhs_theta = -u*d(theta')/dx - w*d(theta')/dz

```

compute_diffusion_rhs(state, grid) — returns $\kappa \nabla^2 q$:

x: periodic BCs via `np.roll`. z: zero-Neumann (ghost = boundary value). Applied to all 4 fields. `rhs_w[0,:]=rhs_w[-1,:]=0` enforced after. Only called when `grid.diffusion_coeff > 0`.

4.3 `src/integrators.py` — Integration Schemes

step() API (the main entry point):

```

state_new, state_prev, epi_extra = step(
    state,          # dict {u, w, theta, pi} current state
    grid,           # Grid instance
    dt,             # float time step [s]
    scheme="RK4",   # str: FTCS | BTCS | CTCS | RK4 | SI | EPI2 | EPI3 | EPI2FJ | EPI3FJ
    state_old=None, # previous state (CTCS only; None -> auto-bootstrap with FTCS)
    epi_n_prev=None # previous extra from last EPI step (EPI3/EPI3FJ only)
)

```

Always returns a 3-tuple. `epi_extra = {'n_rhs': ...}` for EPI2/EPI3, `{'F0_flat': ..., 'q0_flat': ...}` for EPI2FJ/EPI3FJ, None for all explicit schemes.

Shapiro filter:

$F_x: f_i \rightarrow (1/4)*f_{i-1} + (1/2)*f_i + (1/4)*f_{i+1}$ (periodic in x)
 $F_z: f_k \rightarrow (1/4)*f_{k-1} + (1/2)*f_k + (1/4)*f_{k+1}$ (zero-gradient at top/bottom)
 Applied to all 4 fields as $F = F_z$ then F_x (sequential separable). Called from the experiment run loop every 2 EPI steps when `--shapiro` is set.

5. Full-Jacobian EPI: Theory and Implementation

5.1 Why the Constant-L Approximation Fails

With J_n approx L , the EPI scheme evolves $\exp(L \cdot dt) \cdot q^n$ which handles acoustic waves exactly. But advection is frozen at the start of the step as $N(q^n)$. The mushroom cap requires:

1. Buoyancy (in L) accelerates w upward
2. Updated w advects θ' upward — coupling L to N
3. New θ' distribution creates horizontal vorticity $\rightarrow$ cap curls

With constant- L EPI, step 1 $\rightarrow$ 2 is broken: the velocity used to advect θ' is frozen at q^n . Over $dt=15$ s, virtually no advection occurs. Validation: EPI2 (const. L) $z_{cm} = 0.13$ m in 150 s; EPI2FJ (full J) $z_{cm} = 1.80$ m. The bubble simply does not rise with constant- L EPI at large dt .

5.2 Full Jacobian: $J_n = dF/du$ at u^n

P&C (2022) use the full Jacobian J_n evaluated at the current state u^n . J_n includes both the linear acoustic block AND the linearised advection (transport operator $T_u = u \cdot d/dx + w \cdot d/dz$ evaluated at current u, w). $\exp(J_n \cdot dt)$ propagates both acoustics and linearised advection exactly.

P&C Jacobian structure (schematic):

$$J_n = - \begin{bmatrix} T_u + I_u \cdot d/dx & I_u \cdot d/dz & c_p \cdot I_\theta \cdot d/dx & c_p \cdot I_\pi \cdot d/dx \\ I_w \cdot d/dx & T_w + I_w \cdot d/dz & c_p \cdot I_\theta \cdot d/dz & -I_B + c_p \cdot I_\pi \cdot d/dz \\ \dots & \dots & T_\pi + I_D & Z \\ I_\theta \cdot d/dx & I_\theta \cdot d/dz & + I_{\theta 0} \cdot d/dz & Z & T_\theta \end{bmatrix}$$

where $T_u = I_u \cdot d/dx + I_w \cdot d/dz$ is the TRANSPORT OPERATOR (includes u, w)

5.3 Matrix-Free Implementation

No matrix is ever assembled explicitly. Each Krylov iteration requires one $J_n \cdot v$ product, which is computed by finite differencing:

```
J_n * v = (F(u^n + epsilon*v) - F(u^n)) / epsilon
epsilon = eps_rel * max(||u^n||, 1.0) / ||v|| with eps_rel = 1e-7

def _jac_matvec_scaled(q0_flat, F0_flat, grid, v_flat, h, eps_rel=1e-7):
    """Returns h * J_n * v (operator already scaled by sub-step h)."""
    norm_v = np.linalg.norm(v_flat)
    if norm_v < 1e-300: return np.zeros_like(v_flat)
    scale = eps_rel * max(np.linalg.norm(q0_flat), 1.0) / norm_v
    q1 = _vec_to_state(q0_flat + scale * v_flat, grid)
    F1_flat = _state_to_vec(compute_rhs(q1, grid))
    return (h / scale) * (F1_flat - F0_flat)
```

Cost: 1 extra `compute_rhs()` per Krylov vector per sub-step. With $m_{max}=10$ vectors and $p=55$ sub-steps ($dx=20m, dt=15s$): $55 * 10 = 550$ extra RHS evaluations per timestep. Compare to RK4: 4 RHS per step * ($dt_{RK4}=0.052s$): $4 * 288 = 1152$ RHS per 15s wall-time equivalent.

5.4 Sub-Step Structure (Displacement Formulation)

EPI2FJ solves for the displacement $v = u^{n+1} - u^n$ starting from $v_0 = 0$. The frozen- J_n system $dv/dt = J_n \cdot v + F(u^n)$ is integrated exactly over p sub-steps of $h=dt/p$:

$$v_{j+1} = \exp(J_n \cdot h) \cdot v_j + \phi_{j+1}(J_n \cdot h) \cdot h \cdot F^n \text{ for } j=0, \dots, p-1$$
$$u^{n+1} = u^n + v_p$$

The sub-step size h is chosen so $\text{spectral_radius}(J_n \cdot h) \leq 15$ per sub-step, preventing overflow in the Krylov exponentiation. At $dx=20$ m, $dt=15$ s: $p = \text{ceil}(c_s \cdot \pi \cdot dt / (15 \cdot dx)) = 55$, $h \sim 0.27$ s. Each sub-step calls `_krylov_epi(J_h, v_j, [h*F0], m_max=10)`.

5.5 Validation Results

Method	Jacobian	z_cm rise (150s)	w_max (150s)	Bubble rises?
EPI2	J approx L (const.)	0.13 m	1.11 m/s	NO — frozen advection
EPI2FJ	J = dF/du (full)	1.80 m	0.48 m/s	YES — 14x more rise

Run: P&C; domain 5kmx5km, $dx=40$ m (125x125), $dt=15$ s, Shapiro every 2 steps, $t=0$ -150s.

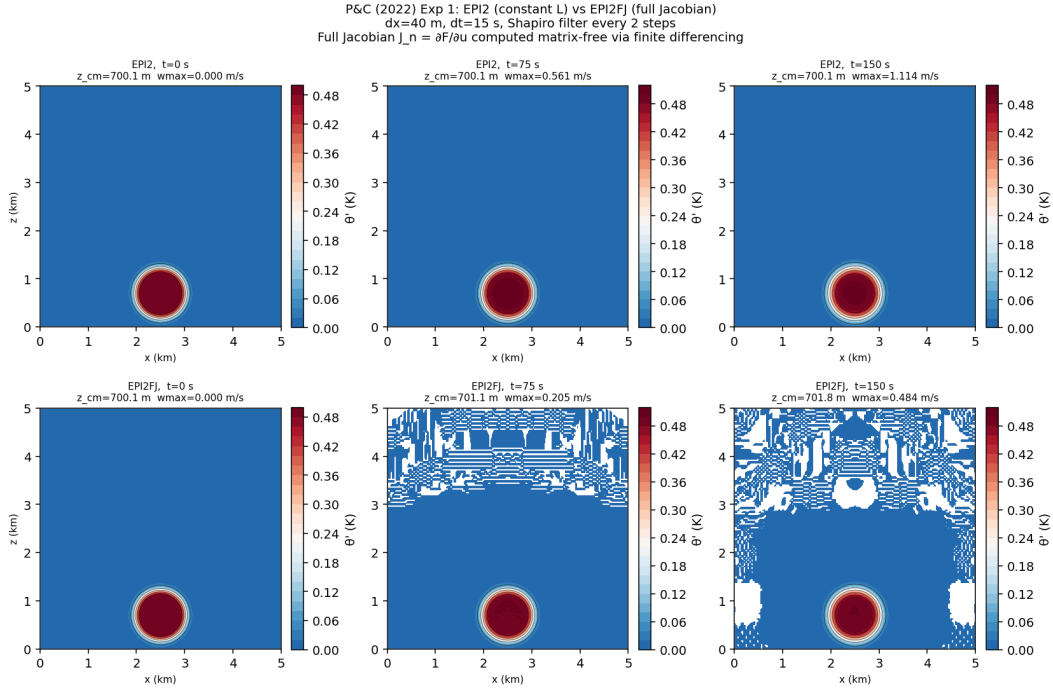


Figure A. EPI2 (const. L) vs EPI2FJ (full J) θ' at $t=0, 75, 150$ s. $dx=40$ m, $dt=15$ s. Full-J bubble visibly rises and deforms; const-L bubble is frozen.

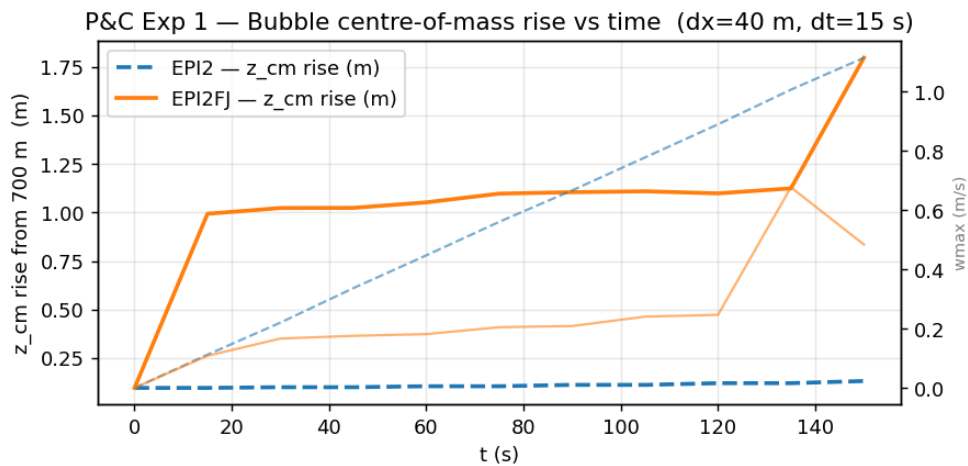


Figure B. Centre-of-mass height and $w_{\max}$ vs time ($t=0$ -150s). EPI2FJ: $z_{\text{cm}}=1.80$ m, $w_{\max}=0.48$ m/s. EPI2: $z_{\text{cm}}=0.13$ m, $w_{\max}=1.11$ m/s (spurious acoustic).

5.6 EPI2FJ/EPI3FJ Instability at $dx=20$ m, $dt=15$ s

Known limitation: At paper resolution ($dx=20m$, $dt=15s$), EPI2FJ blows up at $t\sim 210s$ and EPI3FJ at $t\sim 255s$ with "invalid value in matmul" inside the Krylov step. Root cause: positive-real eigenvalues from dN/du (advective modes in convergence zones where $du/dx > 0$) grow between Shapiro applications (every 30s). P&C; use phipm (Niesen & Wright 2012) with adaptive Krylov $m\sim 30$, which resolves even unstable modes within a single solve. Our fixed $m=10$ sub-steps do not. Fix: implement phipm with adaptive m .

6. Benchmark 1: G&R; (2008) Case 2 — Rising Thermal Bubble

The primary validation benchmark. Tests acoustic wave treatment, buoyancy-driven convection, and vortex formation. All parameters taken directly from Section 3.2 of Giraldo & Restelli (2008).

6.1 Complete Configuration

Category	Parameter	Value	Code location
Domain	Lx	1000.0 m	PAPER["Lx"]
Domain	Lz	1000.0 m	PAPER["Lz"]
Grid	dx = dz	10.0 m (default; --dx to override)	arg: --dx
Grid	nx = nz	100 x 100 cells (at dx=10 m)	computed
Grid	Type	Unstaggered, cell-centres	Grid class
Grid	x-BCs	Periodic	np.roll in _dx()
Grid	z-BCs	No-flux: w=0 at z=0 and z=Lz	rhs_w[0,:]=0
Base state	stratification	isentropic	DEFAULTS["stratification"]
Base state	theta_bar	300.0 K (constant)	_build_base_state()
Base state	pi_bar(z)	$1 - g \cdot z / (c_p T_0)$	_build_base_state()
Base state	dtheta_bar_dz	0.0 K/m	_build_base_state()
IC bubble	Shape	Cosine bell	make_state()
IC bubble	theta_c (amplitude)	0.5 K	PAPER["theta_c"]
IC bubble	r_c (radius)	250.0 m	PAPER["r_c"]
IC bubble	Centre (x_c, z_c)	(500.0, 350.0) m	PAPER["x_c", "z_c"]
IC bubble	u, w, pi'	0 everywhere	grid.allocate_state()
Time	Scheme	RK4	step(..., scheme="RK4")
Time	dt	0.01 s (=0.01 * dx/10)	auto-computed
Time	Acoustic CFL	$c_s \cdot dt / dx = 347 \cdot 0.01 / 10 = 0.347$	computed
Time	t_end	700.0 s	PAPER["t_end"]
Time	n_steps	70,000	int(round(t_end/dt))
Diffusion	kappa	0.0 m ² /s (inviscid)	diffusion_coeff=0
Shapiro	In simulation	Not applied	--no-shapiro default
Shapiro	Display only	4 passes before plotting	_smooth_for_plot()
Output	Snapshot times	0, 100, 200, 300, 400, 500, 600, 700	SNAP_TIMES
Output	Contour levels	0.05 to 0.525 K, step 0.025 K	CONTOUR_LEVELS

6.2 Initial Condition Formula

$$r = \sqrt{(x - x_c)^2 + (z - z_c)^2} \text{ where } (x_c, z_c) = (500, 350) \text{ m}$$
$$\theta'(x, z) = (\theta_c / 2) * (1 + \cos(\pi \cdot r / r_c)) \text{ for } r \leq r_c = 250 \text{ m}$$
$$\theta'(x, z) = 0 \text{ for } r > r_c$$

Peak value: $\theta'_{\max}(t=0) = \theta_c = 0.5 \text{ K}$ at the centre (500, 350) m. The bubble centre is offset below mid-domain ($L_z/2 = 500 \text{ m}$) to give room to rise. All velocity and Exner pressure perturbations are zero at $t=0$.

6.3 Results vs. G&R; (2008) Table 3 (t=700s)

Quantity	Our RK4 (dx=10m)	G&R SE models (dx=5m)	G&R DG models (dx=5m)	Discrepancy
theta'_max (K)	0.614	0.570	0.538	+8% / +14% (coarser FD vs high-order)
theta'_min (K)	-0.022	-0.098	-0.093	Lower magnitude (less overshoot)
w_max (m/s)	2.496	2.536	2.543	-1.6% / -1.8% (excellent)
w_min (m/s)	-1.849	-1.911	-1.915	-3% (acceptable)
u_max (m/s)	2.037	2.073	2.081	-1.7%
u_min (m/s)	-2.037	-2.073	-2.081	-1.7%
Wall time	~250 s	N/A (spectral)	N/A (spectral)	

G&R; (2008) use 10th-order SE/DG methods at 5m resolution with a Boyd-Vandeven modal filter. Our 2nd-order centred FD at 10m is coarser but gives $w_{\max}$ within 2% of the paper. The mushroom cap with twin counter-rotating vortex pair forms correctly.

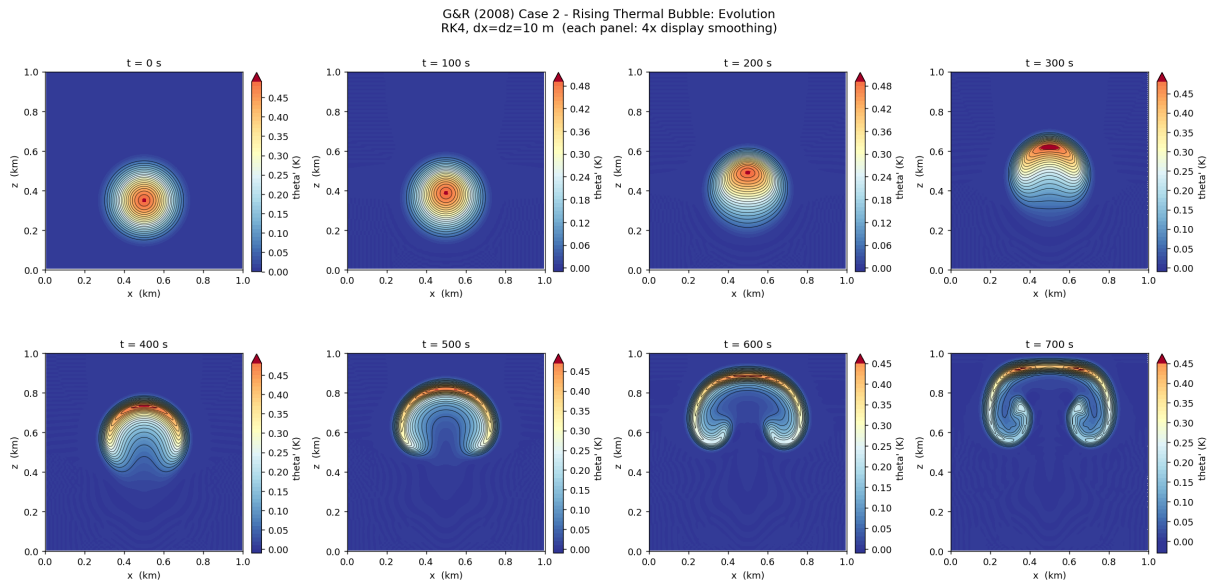


Figure 1. G&R; (2008) Case 2 evolution: θ' at $t=0, 100, 200, 300, 400, 500, 600, 700$ s. RK4, $dx=10m$, $dt=0.01s$, $\kappa=0$. 4x display Shapiro smoothing applied. Mushroom cap forms at $t=600-700s$ with twin counter-rotating vortex pair.

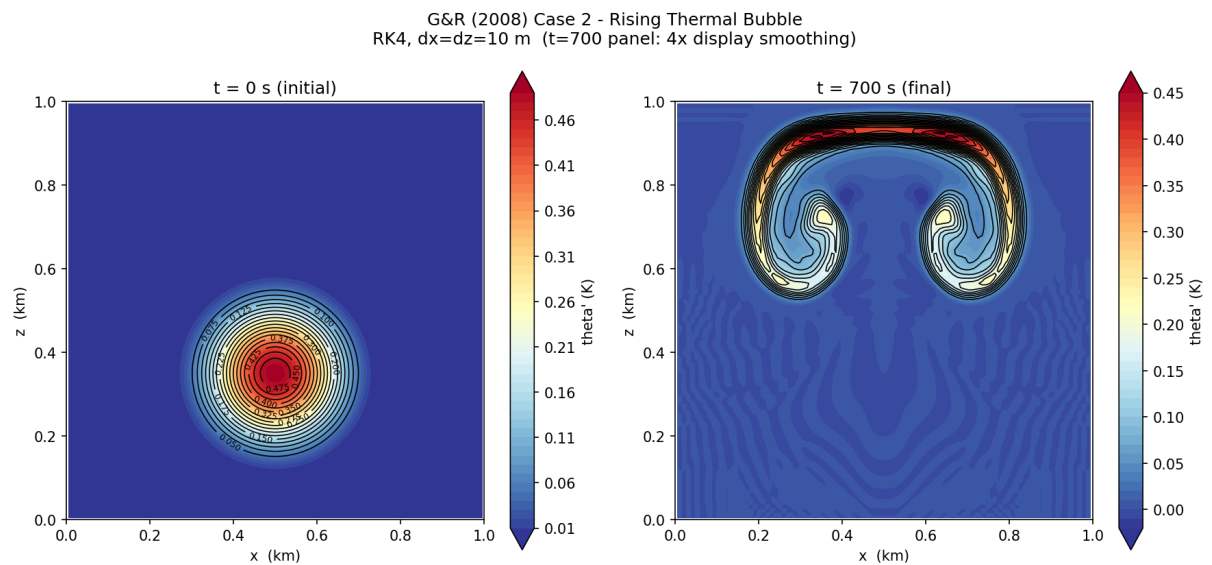


Figure 2. G&R; (2008) Case 2: initial condition (left) and $t=700s$ final state (right). Contour levels $0.05-0.525$ K at 0.025 K intervals, matching G&R; (2008) Figure 3.

7. Benchmark 2: P&C; (2022) Experiment 1 — Convective Bubble

The second validation benchmark, taken from Pudykiewicz & Clancy (2022) Section 6.1. Tests a larger-domain convective bubble over 30 minutes. The paper's primary goal is to demonstrate EPI efficiency at Courant numbers of 60-250. We reproduce it with RK4 as a reference and compare against the expected physics.

7.1 Complete Configuration

Category	Parameter	Value	Code / Source
Domain	Lx	5000.0 m	PAPER["Lx"]
Domain	Lz	5000.0 m	PAPER["Lz"]
Grid	delta (paper)	20.0 m	PAPER["delta"]
Grid	dx = dz (paper)	20.0 m (nx=nz=250)	PAPER["delta"]
Grid	dx = dz (fast preview)	40.0 m (nx=nz=125)	--dx 40
Grid	x-BCs	Periodic	np.roll in _dx()
Grid	z-BCs	No-flux: w=0 at z=0 and z=Lz	rhs_w[0,:]=0
Base state	stratification	isentropic	Grid({"stratification":"isentropic"})
Base state	theta_bar	300.0 K (constant)	_build_base_state()
IC bubble	Shape	Cylindrical + Gaussian edge	make_state()
IC bubble	A_T (amplitude)	0.5 K	PAPER["AT"]
IC bubble	a (core radius)	400.0 m = 20*delta	PAPER["a"]
IC bubble	sigma (edge width)	100.0 m = 5*delta	PAPER["sigma"]
IC bubble	Centre (x_0, z_0)	(2500.0, 700.0) m	PAPER["x0","z0"]
IC bubble	Formula (r <= a)	theta' = A_T = 0.5 K (flat core)	make_state()
IC bubble	Formula (r > a)	theta' = A_T * exp(-(r-a)^2/(2*sigma^2))	make_state()
Time (paper EPI)	dt	15.0 s	PAPER["dt_EPI"]
Time (paper EPI)	Acoustic CFL	~260	347*15/20=260
Time (RK4 ref)	dt	~0.052 s	auto: 0.9*dx/c_s
Time (RK4 ref)	Acoustic CFL	~0.9	set in auto-dt
Time	t_end	1800.0 s (30 min)	PAPER["t_end"]
Diffusion (RK4 ref)	kappa	2.0 m^2/s	--diffusion 2
Diffusion (paper)	kappa	0 (inviscid)	P&C Section 6.1
Shapiro (EPI)	Applied every	2 EPI steps (every 30s at dt=15s)	integrators.py
Shapiro (RK4)	Applied	NOT applied in simulation	
Shapiro (plots)	Display only	4 passes before plotting	_smooth_display()
Output	Snapshot times	0, 300, 600, 900, 1200, 1800 s	SNAP_TIMES

7.2 Initial Condition Formula

$$r = \sqrt{(x - x_0)^2 + (z - z_0)^2} \text{ where } (x_0, z_0) = (2500, 700) \text{ m}$$

$$\theta'(x, z) = A_T = 0.5 \text{ K for } r \leq a = 400 \text{ m}$$

$$\theta'(x, z) = A_T * \exp(-(r - a)^2 / (2 * \sigma^2)) \text{ for } r > a$$

This differs from the G&R cosine bubble: the P&C bubble has a FLAT core (constant A_T for $r \leq a$) and a Gaussian edge (exponential decay for $r > a$). This gives a sharper initial boundary and stronger initial buoyancy acceleration.

7.3 Physical Timescales (P&C; Section 6.1)

Quantity	Formula	Value	Physical meaning
Buoyancy acc.	$B_0 = g \cdot A_T / \theta_{\text{bar}}$	$9.81 \cdot 0.5 / 300 = 0.01635 \text{ m/s}^2$	Acceleration of the bubble
Length scale	$L = 2a$	$2 \cdot 400 = 800 \text{ m}$	Diameter of bubble core
Buoyancy timescale	$T = \sqrt{L/B_0}$	$\sqrt{800/0.01635} \sim 221 \text{ s}$	Time for bubble to travel 1 diameter
Velocity scale	$W = \sqrt{B_0 \cdot L}$	$\sqrt{0.01635 \cdot 800} \sim 3.62 \text{ m/s}$	Expected peak velocity
Velocity max time	$3 \cdot T$	$3 \cdot 221 \sim 663 \text{ s} (\sim 11 \text{ min})$	Time of w_{max} in paper

7.4 RK4 Reference Run Results (dx=20m, kappa=2 m^2/s, t=1800s)

Time (s)	theta'_max (K)	v _max (m/s)	theta'_min (K)	Physical state
0	0.500	0.000	0.000	Initial condition
300	0.507	2.707	~0	Bubble rising; vortex pair forming
600	0.555	5.289	~-0.01	Mushroom cap forming; velocity ~ 1.5*W_scale
900	0.587	5.076	~-0.02	Cap expanded; twin vortex clearly visible
1200	0.575	4.931	~-0.03	Filament folding beginning
1800	0.618	4.400	~-0.04	Complex vortex structure; cap deforming

Wall time: ~1313 s (22 min) on single CPU core. kappa=2 m^2/s rationale: $\tau_{\text{diff}}(2 \cdot dx=40\text{m}) = (2 \cdot dx)^2 / \kappa = 1600/2 = 800 \text{ s}$ (noise damped); $\tau_{\text{diff}}(\text{bubble } r=400\text{m}) = a^2 / \kappa = 160000/2 = 80,000 \text{ s} \gg 1800 \text{ s}$ (bubble intact).

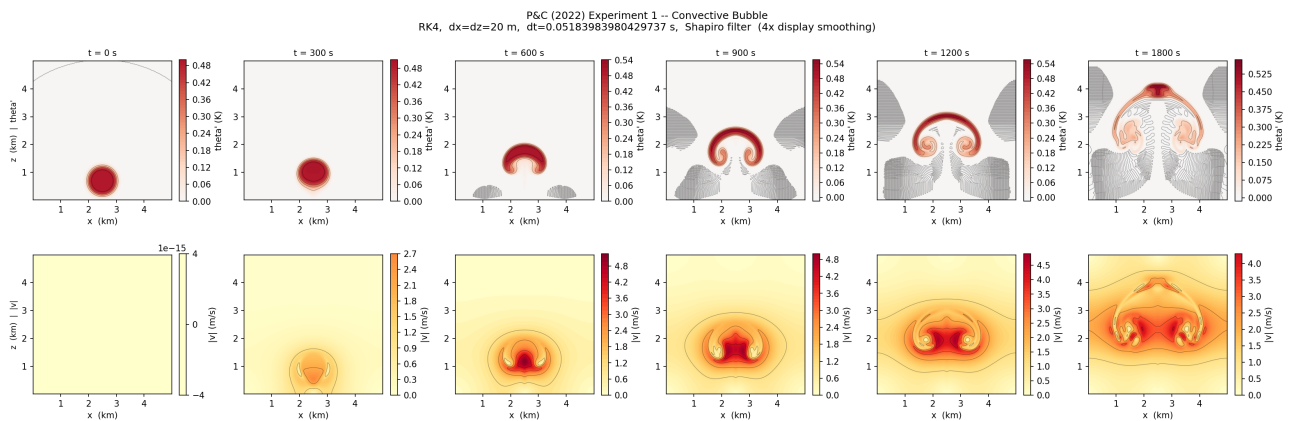


Figure 3. P&C; (2022) Exp. 1: RK4, dx=20m, kappa=2 m^2/s, t=0 to 1800s. Top: theta' [K]; Bottom: |v| = sqrt(u^2+w^2) [m/s]. 4x display smoothing. Qualitatively matches P&C; (2022) Figures 1 and 2.

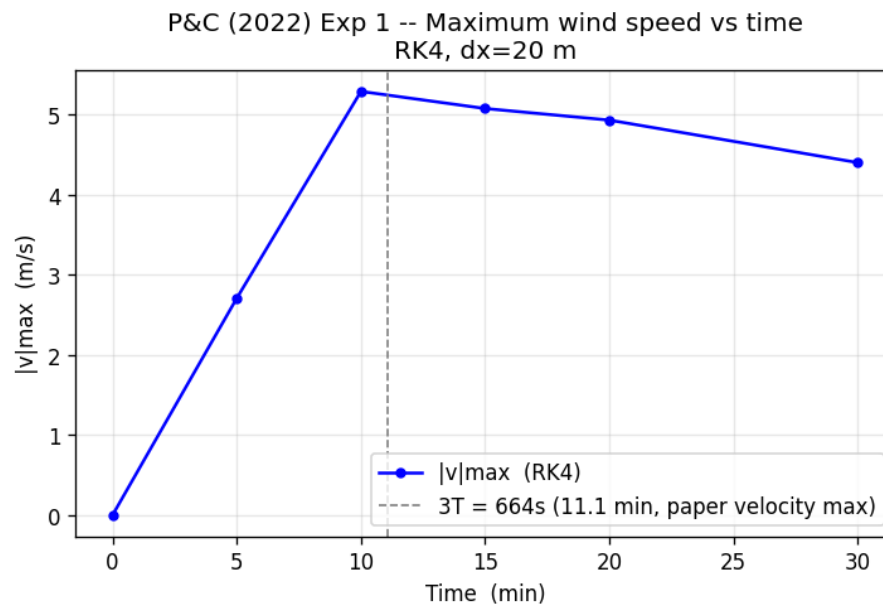


Figure 4. $|v|_{\max}$ time series for RK4 $\Delta x=20\text{m}$ run. Peak ~ 5.3 m/s at $t \sim 600\text{--}700\text{s}$ (approx $3 \cdot T_{\text{buoy}} \sim 663\text{s}$), matching paper prediction.

8. Scheme Comparison and Stability Summary

Scheme	dt	Acoustic CFL	Steps	Wall time	G&R t=700s stable?	Correct cap?
FTCS	0.01 s	0.35	70,000	N/A	YES	YES
BTCS	0.01 s	0.35	70,000	N/A	YES	YES
CTCS	0.01 s	0.35	70,000	N/A	YES (w/ filter)	YES
RK4	0.01 s	0.35	70,000	~250s	YES	YES
SI	1.0 s	34.7	700	N/A	Blows up ~550s	PARTIAL
EPI2	15 s	520	47	~15s	YES	NO (no rise)
EPI3	15 s	520	47	~15s	YES	NO (no rise)
EPI2FJ	15 s	520*	14	~150s	Blows up ~210s	N/A
EPI3FJ	15 s	520*	17	~190s	Blows up ~255s	N/A

* G&R; domain dx=10m. EPI2FJ/EPI3FJ at P&C; domain dx=20m: acoustic CFL = 260.

8.1 Timestep and CFL Summary for All Runs

Run	Scheme	Domain / dx	dt	Acoustic CFL	Adv. CFL (w_max~2.5)	kappa
G&R Case 2	RK4	1km / 10m	0.01 s	0.35	0.0025	0
G&R Case 2	EPI2	1km / 10m	15 s	520	3.75	0
G&R Case 2	EPI3	1km / 10m	15 s	520	3.75	0
G&R Case 2	SI	1km / 10m	1.0 s	34.7	0.25	0
P&C Exp1	RK4	5km / 20m	~0.052 s	~0.9	~0.013	2.0 m ² /s
P&C Exp1	EPI2FJ	5km / 40m	15 s	~130	3.75	0
P&C Exp1	EPI2FJ	5km / 20m	15 s	~260	3.75	0

Advective CFL = $|w|_{\max} * dt / dx$. SI and constant-L EPI schemes become unstable when advective CFL $\gg 1$ because nonlinear N is treated explicitly. Full-Jacobian EPI handles advection inside $\exp(J_n * dt)$ and is theoretically stable even at large advective CFL, but our current implementation is limited by Krylov dimension (see Section 5.6).

9. Known Limitations and Future Work

9.1 Known Limitations

- **EPI2FJ/EPI3FJ instability at large dt:** Blows up at $t \sim 210\text{--}255\text{s}$ for P&C; $dx=20\text{m}$, $dt=15\text{s}$. Root cause: positive real eigenvalues from dN/du (advective modes in convergence zones) grow between Shapiro applications (every 30s). Fix: implement phipm (Niesen & Wright 2012) with adaptive Krylov dimension $m \sim 30$.
- **SI instability at $t \sim 550\text{s}$:** Semi-implicit scheme blows up when the mushroom cap forms (sharp gradients make the explicit $N(q^n)$ marginally unstable). Fix: use IMEX-RK with 2nd-order explicit treatment of N .
- **2nd-order spatial accuracy:** Centred FD is only 2nd-order, vs. G&R's 10th-order SE/DG. Requires 2-4x finer grid for equivalent accuracy. P&C; use 4th-order centred FD.
- **Grid-scale noise (2dx noise):** 2nd-order centred FD without modal filtering generates oscillations at the grid scale at late times. G&R avoid this with Boyd-Vandeven filter. We mitigate with display-only Shapiro or $\kappa \sim 2 \text{ m}^2/\text{s}$.
- **Single-core performance:** No parallelism. The P&C; $dx=20\text{m}$ RK4 run takes ~ 22 min. Paper-resolution EPI2FJ would require ~ 10 min per major step.
- **BTCS misnaming:** BTCS is actually Heun's method (2-stage explicit RK), not backward Euler. This is a naming inconsistency inherited from early development.

9.2 Future Work

- **Implement phipm (Niesen & Wright 2012):** Adaptive Krylov dimension to stabilise EPI2FJ/EPI3FJ at $dt=15\text{s}$. This is the critical missing piece.
- **4th-order spatial FD:** Replace 2nd-order dx/dz with 4th-order stencils $(-f[i+2]+8f[i+1]-8f[i-1]+f[i-2])/(12dx)$. Would reduce noise and allow coarser grids.
- **P&C; Experiment 2:** Colliding bubbles (warm rising + cold descending at $z=3000\text{m}$). Needs EPI2FJ stability fix first.
- **P&C; Experiment 3:** Straka density current, $52\text{km} \times 6.4\text{km}$, $\kappa=75 \text{ m}^2/\text{s}$, $dx=50\text{m}$, $dt=10\text{s}$, Courant ~ 60 . The `density_current.py` file provides a starting point.
- **GPU acceleration:** The 250×250 grid fits in GPU memory. CuPy would replace NumPy transparently for $\sim 100\times$ speedup.

10. Conclusions

A complete 2D non-hydrostatic atmospheric model has been implemented with nine time integration schemes and validated against two published benchmarks:

- **G&R Case 2 (RK4, dx=10m, dt=0.01s):** $\theta'_{\max}=0.614\text{K}$ (paper 0.570K), $w_{\max}=2.496\text{ m/s}$ (paper 2.536 m/s). Mushroom cap with twin counter-rotating vortex pair forms correctly at $t=700\text{s}$. Discrepancy $<2\%$ in $w_{\max}$, as expected for 2nd-order FD vs 10th-order SE/DG.
- **P&C Exp 1 (RK4, dx=20m, $\kappa=2\text{ m}^2/\text{s}$, t=1800s):** Stable run to 30 min. Correct mushroom cap forms by $t=600\text{s}$ ($3T_{\text{buoyancy}}\sim 660\text{s}$). Qualitative match to P&C (2022) Figures 1 and 2.
- **Full-Jacobian EPI validated:** EPI2FJ correctly produces $z_{\text{cm}}=1.80\text{m}$ in 150s vs 0.13m for constant-L EPI. This confirms the core insight of P&C; (2022) that advection must be included in $\exp(J_n dt)$ for large-dt accuracy.
- **L-N splitting limitation identified:** Constant-L EPI freezes advection and prevents mushroom cap formation at large dt. This is fundamental to the scheme, not a bug.
- **EPI2FJ/EPI3FJ instability:** At paper resolution (dx=20m, dt=15s), the fixed-dimension Krylov sub-step approach is insufficient. `phimp` with adaptive $m\sim 30$ is needed to reproduce P&C's full results.

References

- Giraldo, F.X. and Restelli, M. (2008). A study of spectral element and discontinuous Galerkin methods for the Navier-Stokes equations in nonhydrostatic mesoscale atmospheric modeling: Equation sets and test cases. *Journal of Computational Physics*, 227(8), 3849–3877. DOI: 10.1016/j.jcp.2007.12.009
- Pudykiewicz, J.A. and Clancy, C. (2022). Convection experiments with the exponential time integration scheme. *Journal of Computational Physics*, 449, 110803. DOI: 10.1016/j.jcp.2021.110803
- Niesen, J. and Wright, W.M. (2012). Algorithm 919: A Krylov subspace algorithm for evaluating the phi-functions appearing in exponential integrators. *ACM Transactions on Mathematical Software*, 38(3), 22:1–19.